

Objectifs :

1. Savoir concevoir et mettre en œuvre un autopilotage de moteur DC Brushless
2. Savoir utiliser des capteurs magnétiques proportionnels à effet hall (position poignée gaz),
3. Savoir gérer une liaison http entre un PC et un μ C MBED programmé en C++
4. Savoir mettre en œuvre une liaison par bus CAN (déport de la commande de GAZ)
5. Savoir réaliser une petite application graphique de supervision sur une plateforme mobile Android

1 Introduction

L'idée est de réaliser une étude de la commande et du fonctionnement d'un vélo à assistance électrique.

Cette étude comprendra l'analyse du fonctionnement d'un vélo à assistance électrique (VAE) et de ses constituants.

- La programmation en VHDL d'un PLD Altera pour réaliser la commande du collecteur électronique du moteur de VAE
- La programmation en C d'un microcontrôleur MBED qui agira comme superviseur
- La mise en place d'un serveur web et d'une application mobile pour l'affichage des paramètres instantanés du contrôleur de VAE (vitesse, % assistance, jauge énergie, température convertisseur, courant Le Moteur Roue de Véhicule électrique



• Figure 1 : localisation des éléments constitutifs d'une assistance électrique sur un VAE



Moteur brushless 48V 250W (vélo) ou 650W(scooter)

Equipé de 3 capteurs de champs magnétique de la position du rotor

Doté de 8 paires de Poles (pour le moteur de vélo) et de 19 paires de poles (pour le moteur du scooter)

Diametre 16pouces vélo et 40Cm scooter

Le rotor (partie tournante centrale entraînée par la roue) constitue l'inducteur à aimants permanents, et la partie fixe statorique (à la périphérie extérieure du moteur) fixée sur le cadre du vélo contient le bobinage triphasé d'induit ainsi que les 3 capteurs à effets hall qui indiquent la position relative du stator/rotor avec une précision de 60° minimum.

1.1.1 Fonctionnement minimal attendu en fin de BUT2 S4 :

- 1) Assurer l'autopilotage temps réel du moteur brushless à partir de la connaissance de sa position rotorique (à l'aide des informations booléennes des 3 capteurs à effets halls)
- 2) Permettre une commande en vitesse variable depuis soit la console série de gestion soit à partir de l'exploitation de la position angulaire la poignée de gaz/capteur de pédalage de manière instantanée ou progressive (lissage des variations de consigne de poignée de gaz)
- 3) Mesurer la mesure de vitesse de rotation du moteur en m/h (0 à 60000m/h)
- 4) Gérer le capteur de freinage (fonctionnement non réversible en énergie) pour piloter l'assistance électrique.
- 5) Gérer la calibration persistante de la poignée de gaz avec une sauvegarde dans un fichier texte sur le Mbed
- 6) Gérer une limitation en vitesse (bride) réglable depuis la console série et sauvegardée de manière persistante
- 7) Afficher les valeurs instantanées depuis la console série des valeurs de consigne de gaz (0 à 100%), tension batterie (0 à 60V), courant batterie (-5A à +5A), température du convertisseur (-25°C à +100°C)
- 8) Proposer un affichage texte de ces mêmes paramètres depuis une requête issue d'un navigateur web selon le protocole http
- 9) Mettre en place une jauge d'énergie permettant d'estimer la capacité batterie restante en se basant sur les courants absorbés par l'onduleur de puissance.
- 10) Proposer de créer une petite application Android cliente http permettant de collecter les informations servies en http par le microcontrôleur central (utilisation de [appinventor](http://appinventor.com))
- 11) Proposer de déporter la gestion de la consigne d'assistance au pédalage et de freinage via un réseau CAN BUS. La poignée sera alors connectée depuis un 2ème microcontrôleur relié en BUS Can au contrôleur de puissance (réalisation d'un capteur intelligent et communiquant).

1.2 Topologie générale du projet

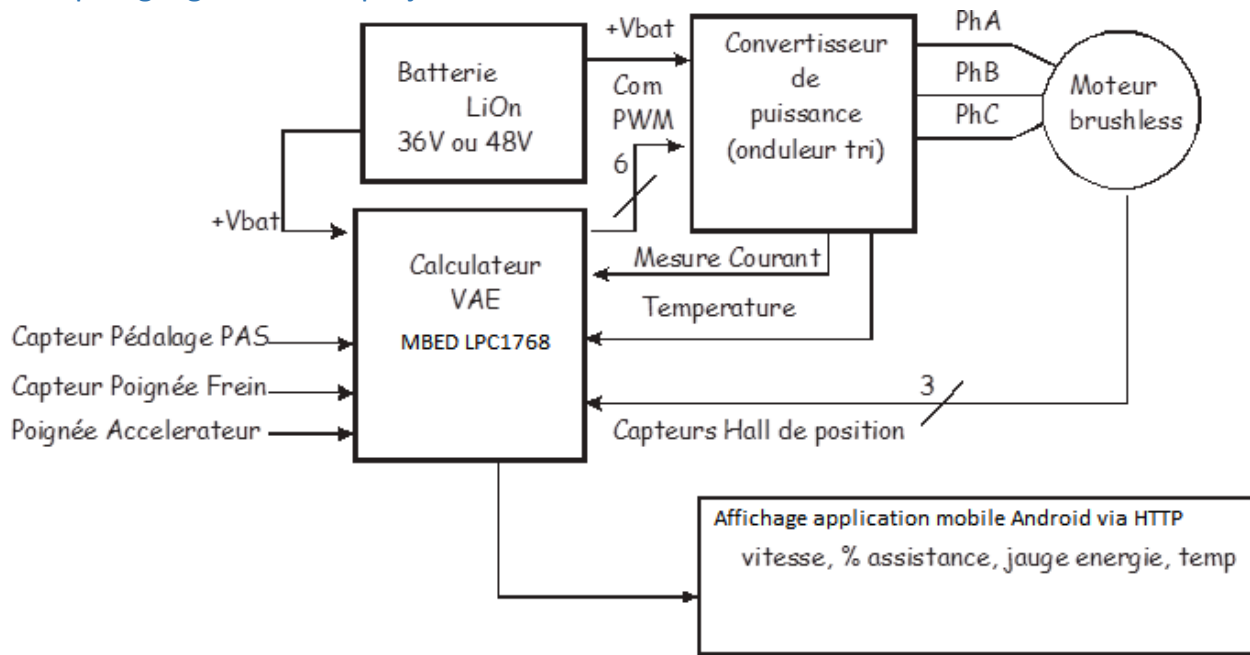


Figure 2 : topologie générale du projet

Vous aurez plus de détails en regardant la fiche ressource disponible sur LMS UNICE dans le cours « SAE ROUE » à savoir la fiche 0 :

https://lms.univcotedazur.fr/2023/pluginfile.php/168430/mod_resource/content/1/Fiche%200%20elements%20constitutifs%20dun%20vae.pdf

Sur la topologie présentée en figure 1, nous avons un contrôleur de VAE programmable constitué de 2 éléments distincts :

- Une carte dotée d'un PLD ALTERA 7064SLC44-10 : cette carte sera chargée de la partie autopilotage temps réel du moteur brushless afin d'assurer l'alimentation des phases de l'induit en fonction de la position angulaire de l'inducteur.
- Une carte MBED dotée d'un LPC1768: cette carte sera chargée de l'interface utilisateur et de la gestion des entrées capteurs freinage (break), sur intensité (overcurrent), défaillance (FLTA) et direction (DIR) ainsi que de la lecture de la poignée de gaz, capteur de température présent dans le convertisseur, tension de la batterie et courant délivré par la batterie. Il aura donc un rôle de superviseur non temps réel de l'autopilotage et fournira à l'utilisateur final deux IHM distincts :
 - Une IHM en mode console (protocole série RS232) pour la maintenance en atelier et le paramétrage primaire du contrôleur de VAE
 - Une IHM en mode serveur WEB (liaison TCP/IP) destinée à fournir un tableau de bord des paramètres de fonctionnement du contrôleur de VAE (indicateur de vitesse, autonomie batterie, puissance poignée de gaz, odomètre....)

Ces deux cartes peuvent échanger entre elles un octet de données en lecture et écriture via une liaison // 8 bits bidirectionnelle

Cet octet de données échangé servira au uC MBED à transmettre vers le PLD Altera une consigne de tension de sortie de l'onduleur (Ref_PWM codée sur 8 bits) , et dans le sens PLD>MBED à remonter un registre d'état de 8bits contenant le numéro de secteur hall, état du frein, indicateur de défaillance, l'indicateur de surintensité et la direction demandée par l'utilisateur.

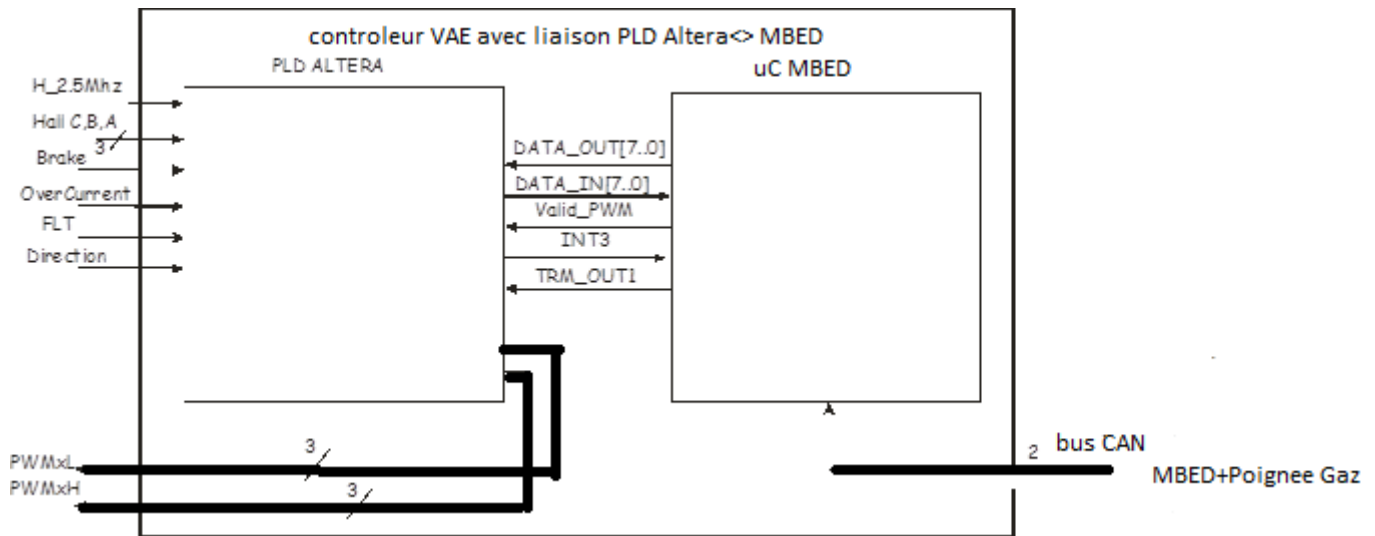


Figure 3 : Contrôleur VAE programmable composé de deux cartes (PLD altera et MBEDLPC1768) pouvant communiquer entre elles grâce à un bus // de 8 bits de données

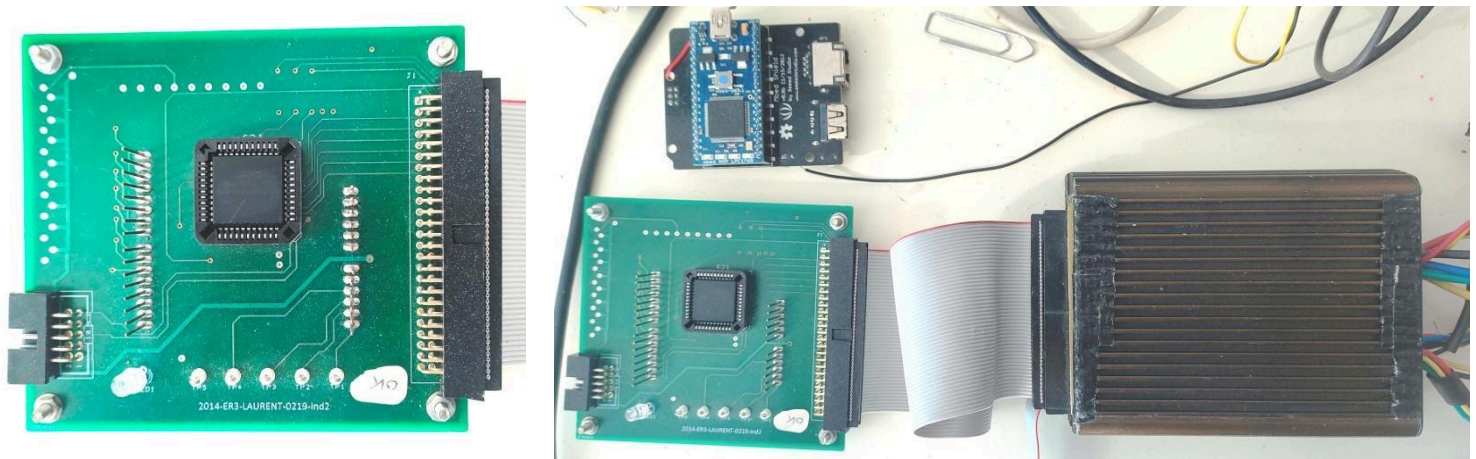


Figure 4: carte Max7000S utilisée en SAE roue(gauche) et ensemble des cartes de la SAE Roue VAE avec l'onduleur (droite)

Objectif général du projet : Il s'agit donc de pouvoir piloter le moteur roue de technologie Brushless (moteur courant continu sans balais) en commande trapèze et en vitesse variable à partir d'un microcontrôleur MBED et d'un circuit logique programmable MAX7064SLC44-10.

Le microcontrôleur sera chargé de la supervision non-temps réel de la commande du moteur (variation de vitesse, communication avec l'utilisateur, mesure des paramètres physiques...)

Le PLD altera sera chargé de l'autopilotage temps réel en vitesse variable du moteur (la boucle d'autopilotage visible sur la figure2 est en effet critique et présente des contraintes temps réel dures).

1.3 Tableau et affectation générale des signaux échangés entre le MBED et le PLD Altera

Pin sur PLD Altera.	Broche connecteur HE10 50p sur la carte MBED	Localisation du les vecteurs DATA_IN et DATA_OUT dans PLD	Broches sur le MBED LPC1768	Désignation du signal
		DATA_OUT[7..0]	[p24(msb),P23,P10,P9,P8 ,P7,P6,P5(lsb)]	REF_PWM[7..0]
39	13	DATA_IN0	--	HALLA
40	14	DATA_IN1	--	HALLB
41	15	DATA_IN2	--	HALLC
36	12	DATA_IN3	--	DIRECTION (1=AV 0=AR)
37	11	DATA_IN6	--	OVER_CURRENT* (actif=0)
6	18	DATA_IN4	--	FLTA (actif=1) PLD >uC
5	17	DATA_IN5	--	BRAKE* (actif =0)
33			P21	VALID_PWM (actif=1) uC>PLD
34				--
31	--		P22	Valid_Hall PLD>uC
43	25		--	H_2.5MHZ >PLD
16	24			PWMAL PLD>control
14	23			PWMAH PLD>control
12	22			PWMBL PLD>control
11	21			PWMBH PLD>control
9	20			PWMCL PLD>control
8	19			PWMCH PLD>control
29	--	RD_WRn	P25	RD_WRn uC>PLD
28	--	PCS6	P26	PCS6 uC>PLD
	4	Vgaz	P17	V GAZ/PEDALAGE (analog)
	5	Vbat	P18	VBAT (analog)
	6	Vtemp	P19	VTemp (analog)
	7	Vldc	P20	Vlmes (analog)
			P29	CAN TX2 bus can
			P30	CAN RX2 bus can

Figure 5 : tableau générale d'affectation des signaux entre le PLD et le uC Mbed LPC1768

Note : Sur la carte [MBED shield](#) figure des connecteurs de liaison GROVE notés

J10 : RX1=P14, TX1=P13

J7 : RX2/SCL2 :P27 TX2/SDA2 :P28

J13 :RX3/SCL1 :P10 en conflit avec le port // 8bits réservé, TX3/SDA1 :P9 en conflit avec le port // 8bits réservé

J16 :ADC0 :P15, ADC1 :P16

Led locales disponibles sur le module LPC1768

```

DigitalOut led1(LED1); //available
DigitalOut led2(LED2); // available
DigitalOut led3(LED3); // blink when can message is sent
DigitalOut led4(LED4); // blink when can message is received

```

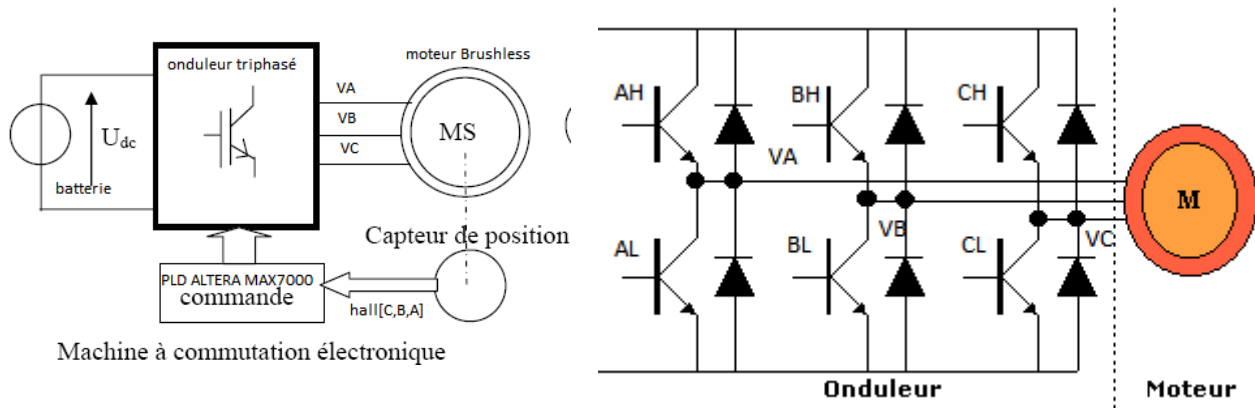


Figure 6: architecture de l'autopilotage (gauche) et détail de l'onduleur (droite)

La Partie puissance est assurée par un onduleur triphasé préfabriqué (boite noire voir sur la figure 1) sur lequel nous avons extrait et dirigé les 6 commandes de transistor des bras d'onduleur vers le PLD ALTERA à savoir AH/AL, BH/BL, CH/CL (c'est le PLD qui assurera donc le commande des 3 bras d'onduleur) à partir de la connaissance de la position instantanée du rotor (inducteur) du moteur.

A partir de là il élabore les 3 consignes de tension VA,VB,VC en fonction de la position actuelle du rotor (capteurs Hall[C,B,A]) et de la consigne de vitesse que désire l'utilisateur (Référence PWM au format 8 bits // fournis par le uC MBED au PLD ALTERA).

2 Principe d'autopilotage du moteur synchrone brushless en commande trapèze via le PLD altera.

Voici une vidéo de vulgarisation du principe de commande d'un moteur brushless a commande trapeze :

(texas instruments) <https://www.youtube.com/watch?v=xIORHY8li90>

Il y a en fait 6 états possibles d'alimentation du stator du moteur brushless en fonction des 6 positions différentes mesurables par les 3 capteurs de position à effets hall du Rotor (HALL[CBA], les positions « 000 » et « 111 » étant impossibles ou équivalentes à une défaillance des capteurs du moteur). Plus de détail en utilisant la fiche ressource disponible sur LMS Unice cours SAE Roue à savoir la fiche 1.



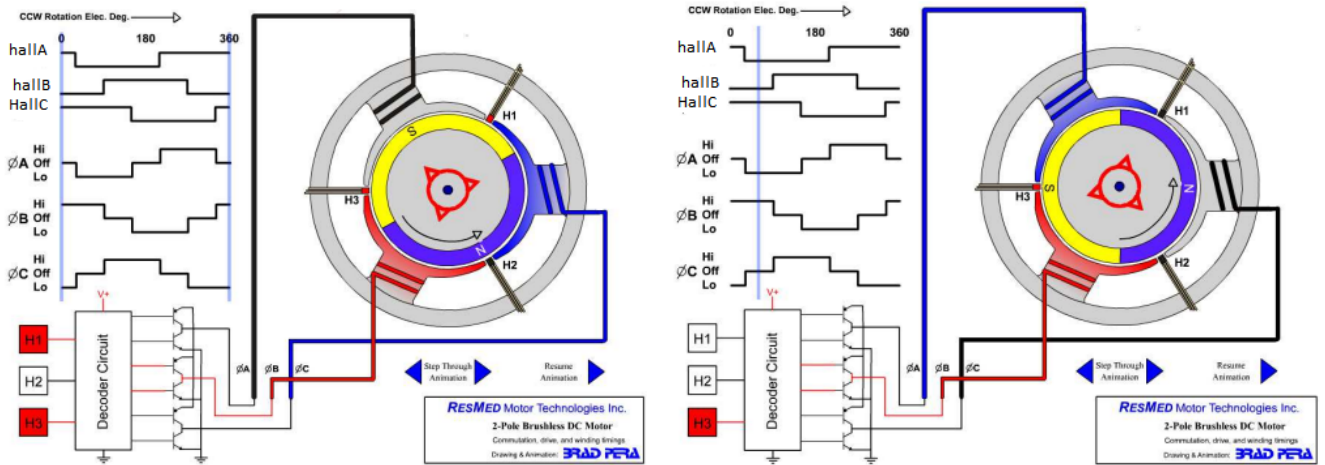


Figure 7: Par convention le capteur H3 sera le MSB et le capteur H1 sera le LSB de la position lue. On représente la position angulaire "101" $\Leftrightarrow$ 5 (figure de gauche avec Position 6) et la position angulaire « 100 » $\Leftrightarrow$ 4 sur la figure de droite avec Position 2 et Phase A-

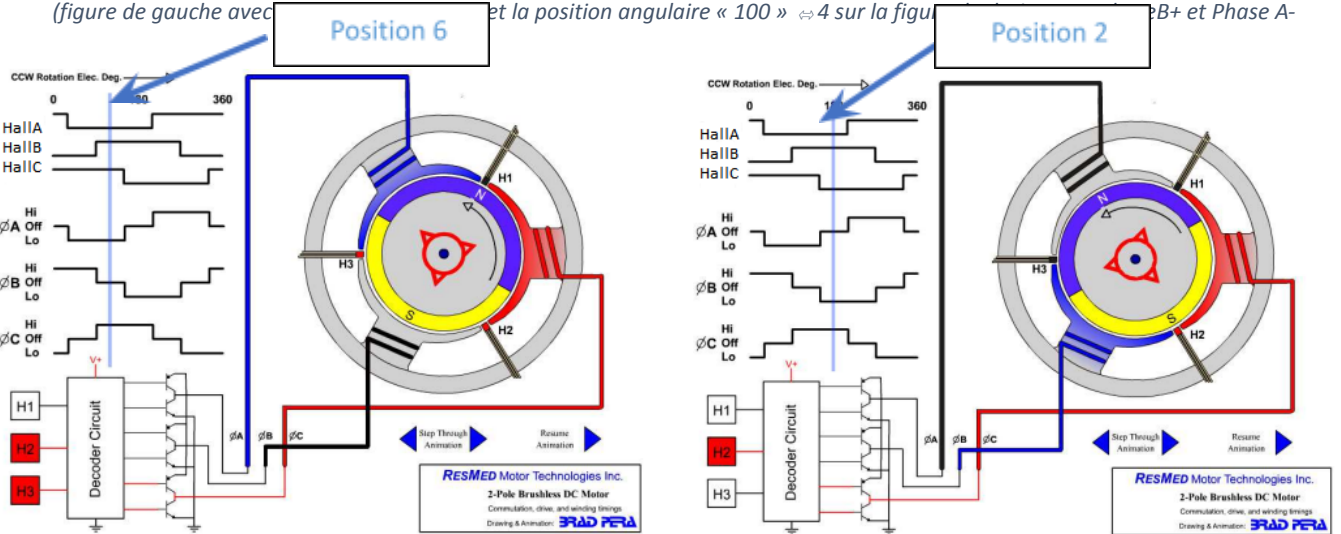


Figure 5: position angulaire "110" $\Leftrightarrow$ 6 (gauche) et « 010 » $\Leftrightarrow$ 2 à droite

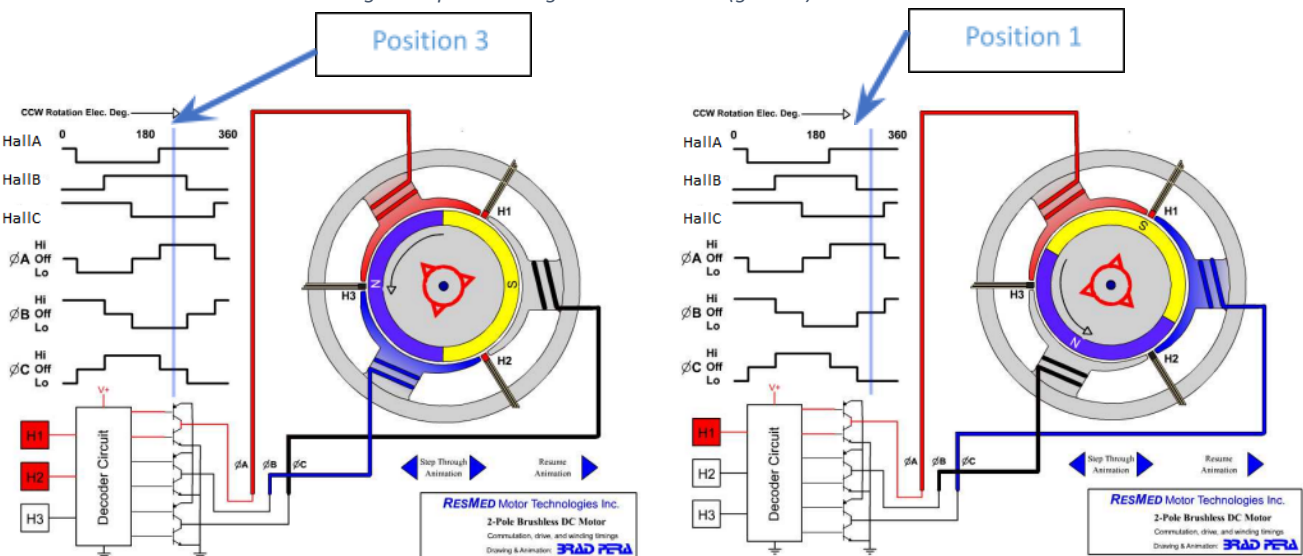


Figure 6: position angulaire "011" $\Leftrightarrow$ 3 (gauche) et « 001 » $\Leftrightarrow$ 1 à droite

Comme le montrent les figures 4,5 et 6, la commande des 6 transistors de l'onduleur doit être « SYNCHRONES » de la position angulaire instantanée du rotor (c'est ce que l'on appelle l'autopilotage du brushless). Cette fonction est en fait l'équivalent d'un collecteur numérique (sa version numérique existe dans le moteur à courant continu classique).

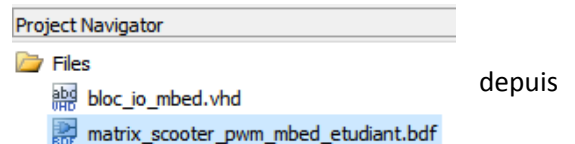
Plus de détails ici :

https://lms.univ-cotedazur.fr/2023/pluginfile.php/168431/mod_resource/content/1/fiche%20n1%20principe%20d%20autopilotage%20dun%20moteur%20brushless.pdf

3 Contrôle temps réel de l'autopilotage via le PLD ALTERA :

Préliminaires ouverture du projet Quartus PLD_mbed_etudiant :

Télécharger le dossier projet_altera_sae_roue.zip depuis LMS le cours sae roue et décompresser dans votre dossier z:\sae_roue\



Démarrer **Quartus II v13.0.1** (icône sur le bureau **et non pas la quartus17**).

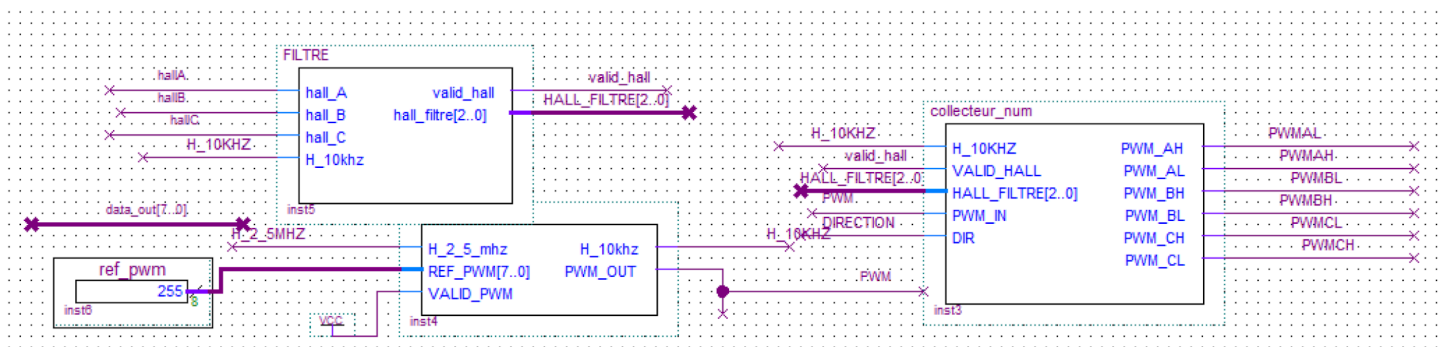
Ouvrir un projet existant *menu File>open projet> PLD_mbed_etudiant*

Le projet contient actuellement 2 fichiers sources bdf

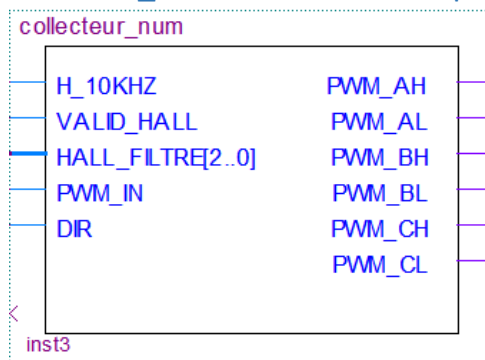
Bloc_io_mbed.vhd : bibliothèque permettant un dialogue bidirectionnel 8 bits entre le PLD altera et le microprocesseur MBED LPC1768.

Matrix_scooter_pwm_mbed_etudiant qui est le fichier d'interconnexion général des blocs que nous allons créer en logique programmable au cours de ce TP.

Assemblage futur des blocs à créer en langage VHDL :



3.1 TP3_1 :Collecteur numérique du moteur brushless



Rôle : Ce bloc permet de commander les 6 transistors de l'onduleur triphasé. Les transistor du Bas PWMxL sont commandés en mode pleine onde et réalisent la fonction de collecteur numérique. Les sorties PWMxH distribue la consigne de PWM reçue à l'entrée en direction des transistors situés en haut de l'onduleur.

Ressources d'entrées :

- **Hall_FILTRE[2..0]**: Ce bloc reçoit l'information sur la valeur des 3 capteurs hall. Correspondant à HallC, HallB, HallA. En fonction de cette information, ce bloc pilote les 6 transistors du bas PWMxL et distribue la MLI sur l'un des 3 transistors du Haut PWMxH.
- **VALID_HALL** : Le signal valid_hall indique à quel moment les capteurs de halls présentent une valeur stable. (Actif à 1)
- **DIR** : permet de choisir le sens de rotation de la machine. 1=Marche <AV et 0=Marche Arrière.
- **H_10Khz** : horloge de synchronisation générale 10Khz disponible en interne.
- **PWM_IN** : entrée de PWM modulée en largeur d'impulsion suivant la consigne de pédalage ou de gaz fréquence 10kHz.

Ressources de sortie :

- **PWMxL et PWMxH** : commandes logique des transistors du haut (xH) et du bas (xL) de l'onduleur.

Simulations de fonctionnement attendues :

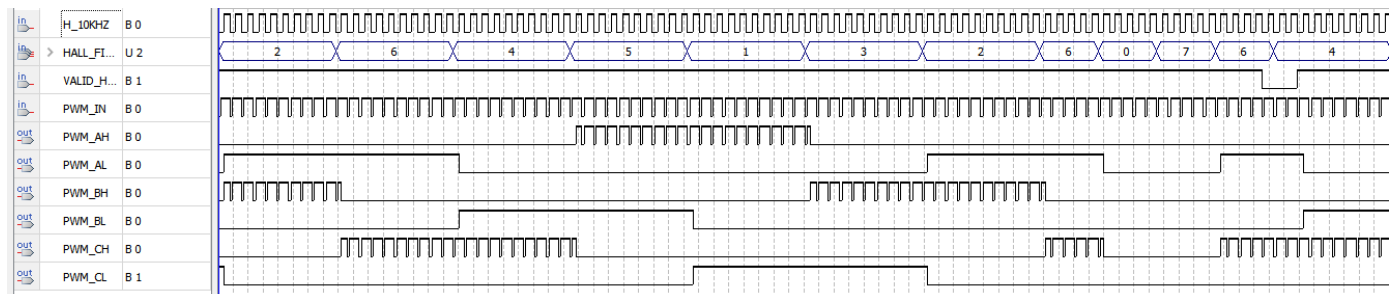


Figure 3-1 : chronogramme de fonctionnement du bloc collecteur_num en marche Avant

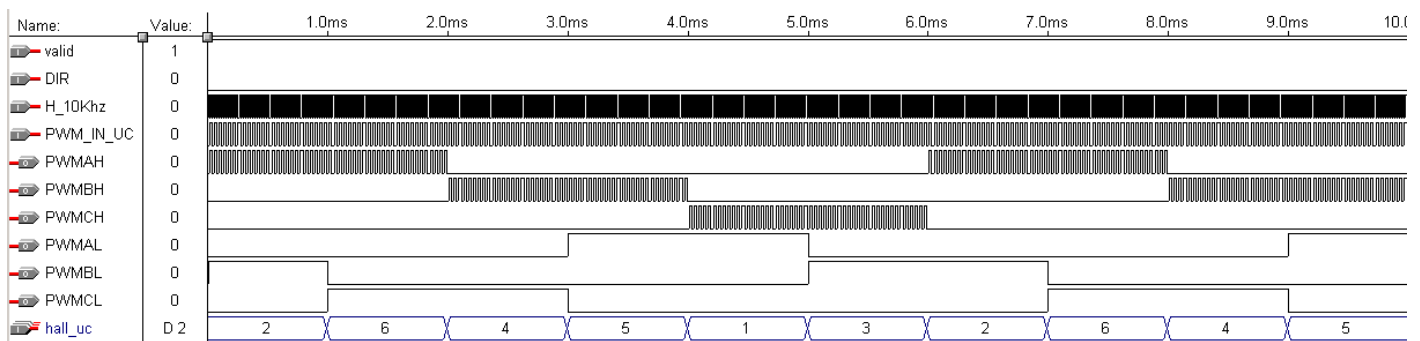


Figure 3-2 : chronogramme de fonctionnement du bloc collecteur_num en marche AR

On n'implémentera que le fonctionnement en marche avant. Si vous êtes en avance vous pourriez rajouter la gestion de la marche arrière.

Créer le fichier source VHDL collecteur_num.vhd et le mettre en top level entity puis le compléter en respectant scrupuleusement les noms des E/S déclarées dans l'entité, le compiler et le simuler. On fournit un squelette de fichier de vecteurs de test collecteur_num.vwf.

Insérer ici votre code vhd de collecteur_num.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.NUMERIC_STD.ALL;
```

```
entity collecteur_num is
```

```
    Port (
```

```
        HALL_FILTRE : in STD_LOGIC_VECTOR(2 downto 0);
```

```
        VALID_HALL  : in STD_LOGIC;
```

```
        DIR         : in STD_LOGIC;
```

```
        PWM_IN      : in STD_LOGIC;
```

```
        H_10KHZ     : in STD_LOGIC;
```

```
        PWM_AH, PWM_AL, PWM_BH, PWM_BL, PWM_CH, PWM_CL : out STD_LOGIC
```

```
    );
```

```
end collecteur_num;
```

```
architecture archi of collecteur_num is
```

```
    signal sig : std_logic_vector (5 downto 0);
```

```
begin
```

```
    process(H_10KHZ, VALID_HALL)
```

```
begin
```

```
if (H_10KHZ'event and H_10KHZ = '1' and VALID_HALL = '1') then
```

```
    case HALL_FILTRE is
```

```
        when "001" => sig <= "100001"; --AH,CL
```

```
        when "010" => sig <= "011000"; --AL,BH
```

```
        when "011" => sig <= "001001"; --BH,CL
```

```
        when "100" => sig <= "000110"; --BL,CH
```

```
        when "101" => sig <= "100100"; --AH,BL
```

```
        when "110" => sig <= "010010"; --AL,CH
```

```
        when others => sig <= "000000";
```

```
    end case;
```

```
end if;
```

```
end process;
```

```
PWM_AH <= sig(5) and PWM_IN;
```

```
PWM_AL <= sig(4);
```

```
PWM_BH <= sig(3) and PWM_IN;
```

```
PWM_BL <= sig(2);
```

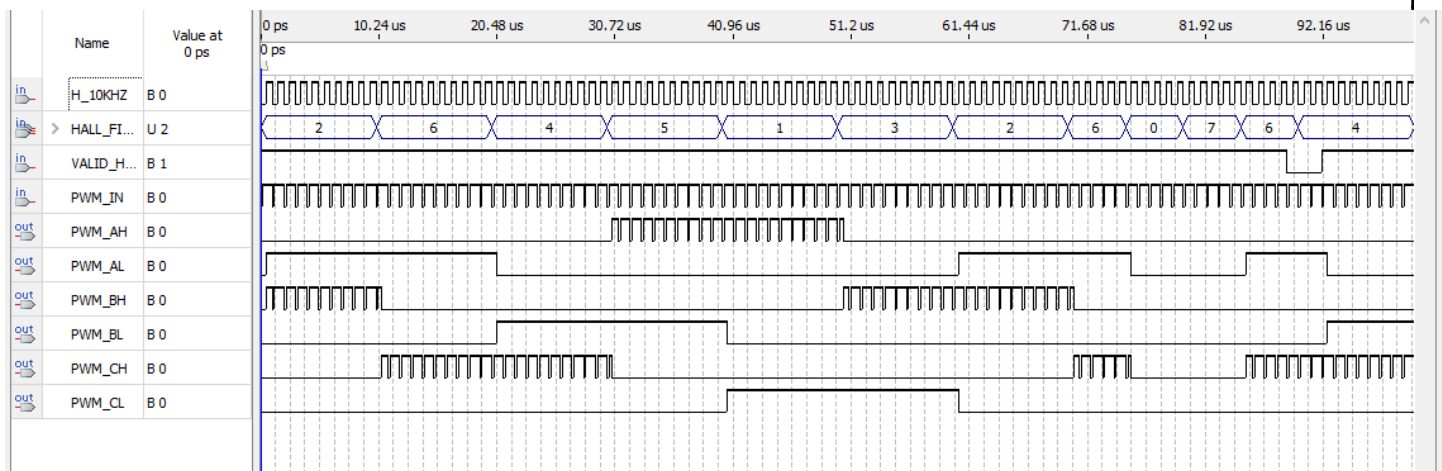
```
PWM_CH <= sig(1) and PWM_IN;
```

```
PWM_CL <= sig(0);
```

```
end archi;
```

- Ouvrir le squelette de simulation collecteur_num.vwf et lancer une simulation fonctionnelle. Comparez aux résultats attendus

Collez ici votre simulation de collecteur_num.vwf

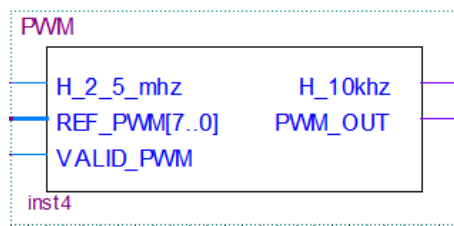


- Analysez soigneusement la commande des 6 transistors
- Observez en particulier le comportement du système lorsque Hall[c,b,a] = 0 ou à 7
- Observez en particulier lorsque l'entrée de validation des capteurs hall (Valid_Hall) devient inactive.

Création du symbole équivalent à collecteur_num.vhd.

- Sélectionner le fichier collecteur_num.vhd et créer son symbole par défaut **collecteur_num.bsf**
- Ouvrir **matrix_scooteur_pwm_etudiant.bdf** et y connecter le nouveau symbole **collecteur_num.bsf** et le connecter aux E/S présentent (on remplacera le symbole déjà existant dans ce fichier d'assemblage).

3.2 TP3_2 :générateur de signaux PWM



Dans notre projet, le uC mbed doit pouvoir envoyer au pld un mot de 8 bits // représentant une consigne de PWM (Ref_PWM[7..0]) variant entre 0 et 255 qui permettra de moduler en largeur d'impulsion la commande des trois transistors du haut de l'onduleur à savoir PWM_AH, PWM_BH et PWM_CH.

La fréquence de cette PWM sera d'environ de 10Khz et nous partirons pour ce faire une horloge rapide présente sur la maquette H_2_5Mhz qui oscille à 2,5Mhz.

Rôle : Ce bloc reçoit la consigne de PWM sur 8 bits en // en provenance du microcontrôleur et fournit un signal de PWM de rapport cyclique variable et de fréquence de découpage de 10Khz PWM_OUT sur un bit. Ce signal sera plus tard distribué par le bloc collecteur numérique précédent. Il génère également une horloge interne de 10Khz environ à destination des autres blocs fonctionnels

Ressources d'entrée :

- **H2.5Mhz** : horloge rapide environ 2.5Mhz (présente dans la maquette)
- **Valid_PWM** : signal de validation global de la PWM (actif à 1) permettant de prévoir un arrêt d'urgence ou une limitation en courant par détection de sur intensité. Si Valid_PWM=0, la sortie PWM_OUT=0
- **PWM_REF[7..0]** : consigne PWM d'entrée fournie par le uC MBED.

Ressources de sortie :

- **H_10Khz** : sortie horloge 10Khz pour usage interne dans le PLD.
- **PWM_OUT** : sortie PWM modulée en fonction de PWM_REF[7..0]

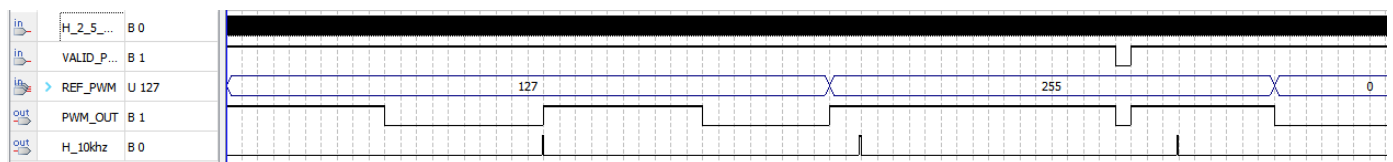


Figure 10 : Exemple de Fonctionnement attendu du bloc générateur PWM

- Créer un fichier source VHDL **PWM.vhd** fournis et saisir la description comportementale correspondant au cahier des charges.
- Mettre **PMW.bdf** en TopLevelEntity et lancez une compilation fonctionnelle. Corrigez les erreurs éventuelles.
- Ouvrir le squelette de simulation **PWM.vwf** fournis et lancer une simulation fonctionnelle. Comparez aux résultats attendus
- Créer le symbole **PWM.bsf** équivalent à la fonction **PWM.bdf**

Collez ici votre code VHDL de **PWM.vhd**

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;
```


entity PWM is

Port (

H_2_5_mhz : in STD_LOGIC;

REF_PWM : in integer range 0 to 255;

VALID_PWM : in STD_LOGIC;

H_10khz : out STD_LOGIC;-- horloge

PWM_OUT : out STD_LOGIC-- sortie modulée en fonction de PWM_REF

);

end PWM;

architecture archi of PWM is

signal sig : integer range 0 to 255; --declaration compteur

begin

process (H_2_5_mhz)

begin

if(H_2_5_mhz'event and H_2_5_mhz='1')then

if(sig=255)then

sig<=0;

H_10khz<='1';

```
else
    sig <=sig+1;
    H_10khz<='0';
end if;

if(VALID_PWM='1') then

    if (REF_PWM>=sig) then
        PWM_OUT<='1';

    else
        PWM_OUT<='0';
    end if;

    else pwm_OUT<='0';

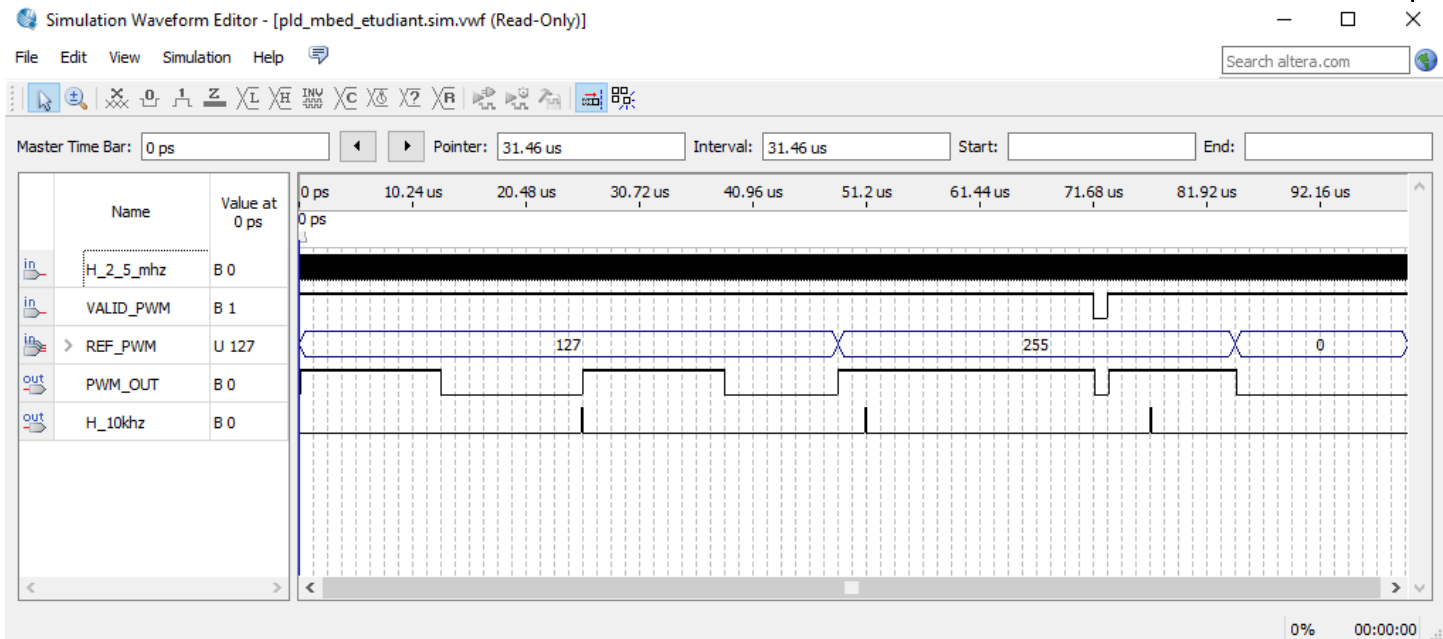
end if;

end if;

end process;

end archi ;
```

Collez ici votre simulation **pwm.vwf**



- Vérifiez la cohérence entre la référence de REF_PWM et le rapport cyclique du signal PWM_OUT
- Observez en particulier l'effet du signal Valid_PWM sur la sortie PWM_OUT
- Observez les impulsions d'hologe H_10Kz

3.3 TP3_3 : filtrage de la position angulaire du rotor mesurée par les capteurs à effets hall

Dans le cas d'un moteur brushless de véhicule électrique, pour les petites puissances (<3KW) il est courant d'avoir les trois signaux logiques issus des capteurs de position à effet hall qui transistent dans la même gaine électrique que les phases d'alimentation de puissance.

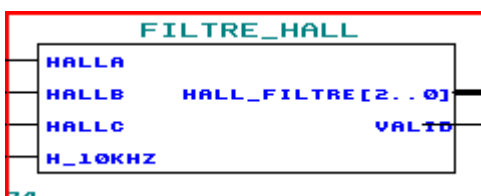
Du fait de cette proximité, par couplage capacitif entre ces fils, il est courant d'observer, sur les signaux logiques hall[CBA], des parasites transmis par couplages électromagnétiques au moment des découpages de tension sur les fils de phase d'alimentation. Ce couplage capacitif parasite a d'autant plus d'effets indésirables que les dV/dT observés au niveau des alimentations de phases sont élevés (cas d'un onduleur à commutation dure et rapide) ce qui est notre cas.

Il faut donc filtrer le signal en provenance des capteurs halls pour diminuer l'effet de ces parasites.

L'algorithme de filtrage est très simple car il s'agit du principe du registre à coïncidence. L'idée est de mémoriser les deux états précédents (sur deux périodes d'horloge H_10khz) des valeurs de Hall[C,B,A] et de les comparer à sa valeur actuelle.

Si la condition $\text{hall}[CBA]_{n-2} = \text{hall}[C,B,A]_{n-1} = \text{hall}[C,B,A]_n$ alors cela signifie que la valeur actuelle $\text{hall}[C,B,A]_n$ est stable depuis deux périodes d'horloge et donc quelle est fiable. Dans ce cas on présente cette nouvelle valeur sur la sortie du bloc ($\text{hall_filtre}[C,B,A]$) et on indique que cette valeur est exploitable ($\text{Valid_Hall}=1$).

Sinon on maintient en sortie la dernière valeur stable connue de Hall[C,B,A] et on désactive $\text{Valid_Hall}=0$.



Rôle : ce bloc permet de filtrer numériquement les signaux issus des capteurs hall et qui peuvent être bruités par les signaux de puissance au moment de leur changement d'état : L'idée du filtrage est de comparer le signal hall sur 2

périodes d'horloge de base consécutives et de ne valider le signal de sortie (Valid) que lorsque le signal hall d'entrée est resté constant pendant au moins 2 périodes d'horloges consécutives.

Ressources d'entrée :

- **Hall A, B et Hall C** sont les 3 capteurs halls de position.
- **H_10Khz**, horloge de base interne de 10Khz.

Ressources de sortie :

- **Hall_FILTRE[2..0]**. Regroupement de Hall C, B et A après filtrage.
- **Valid** : indicateur de signal stable de l'état des capteurs hall (actif à 1).

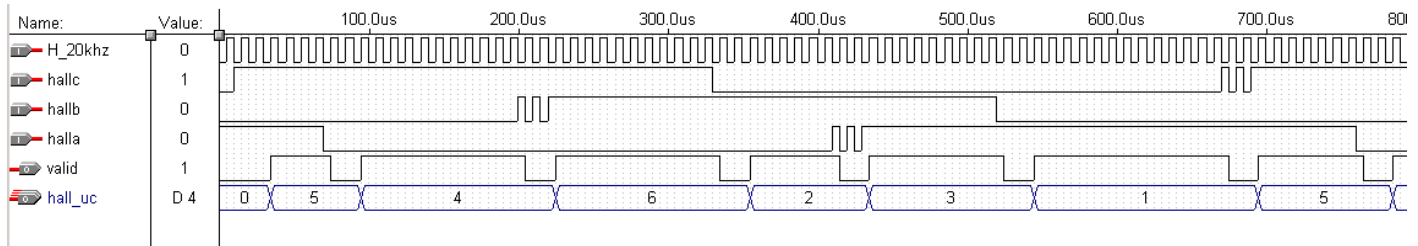


Figure 2-13 : chronogramme de fonctionnement du filtre de capteur hall.

- Créer le fichier source VHDL **FILTRE.vhd** et saisir la description comportementale correspondant au cahier des charges .
- Mettre **FILTRE.vhd** en TopLevelEntity et lancez une compilation fonctionnelle. Corrigez les erreurs éventuelles.
- Ouvrir le squelette de simulation **FILTRE.vwf** fournis et lancer une simulation fonctionnelle. Comparez aux résultats attendus
- Créer le symbole **FILTRE.bsf** équivalent à la fonction **FILTRE.vhd** et l'assembler dans le fichier général d'interconnexions **matrix_scooter_pwm_mbed_etudiant.bdf**

Collez ici votre code source de FILTRE_HALL.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
```

entity FILTRE is

Port (

hall_A : in STD_LOGIC; -- capteur hall de position A

```
hall_B      : in STD_LOGIC; -- capteur hall de position B

hall_C      : in STD_LOGIC; -- capteur hall de position C

H_10khz     : in STD_LOGIC;-- horloge de base interne

valid_hall  : out STD_LOGIC;-- indicateur de signal stable

hall_filtre : out STD_LOGIC_VECTOR (2 downto 0) --hall A,B,C après filtrage

);

end FILTRE;

architecture archi of FILTRE is

    signal hall_1 : STD_LOGIC_VECTOR(2 downto 0);
    signal hall_2 : STD_LOGIC_VECTOR(2 downto 0);
    signal hall_3 : STD_LOGIC_VECTOR(2 downto 0);

begin

    hall_1<=hall_C & hall_B & hall_A;

    process(H_10khz)
    begin

        if(H_10khz'event and H_10khz='1') then
            hall_3<=hall_2; --adaptation des valeurs
            hall_2<=hall_1;
```

```
if(hall_3=hall_2 and hall_2=hall_1) then
```

```
    valid_hall<='1';
```

```
    hall_filtre<=hall_1;
```

```
else
```

```
    valid_hall<='0';
```

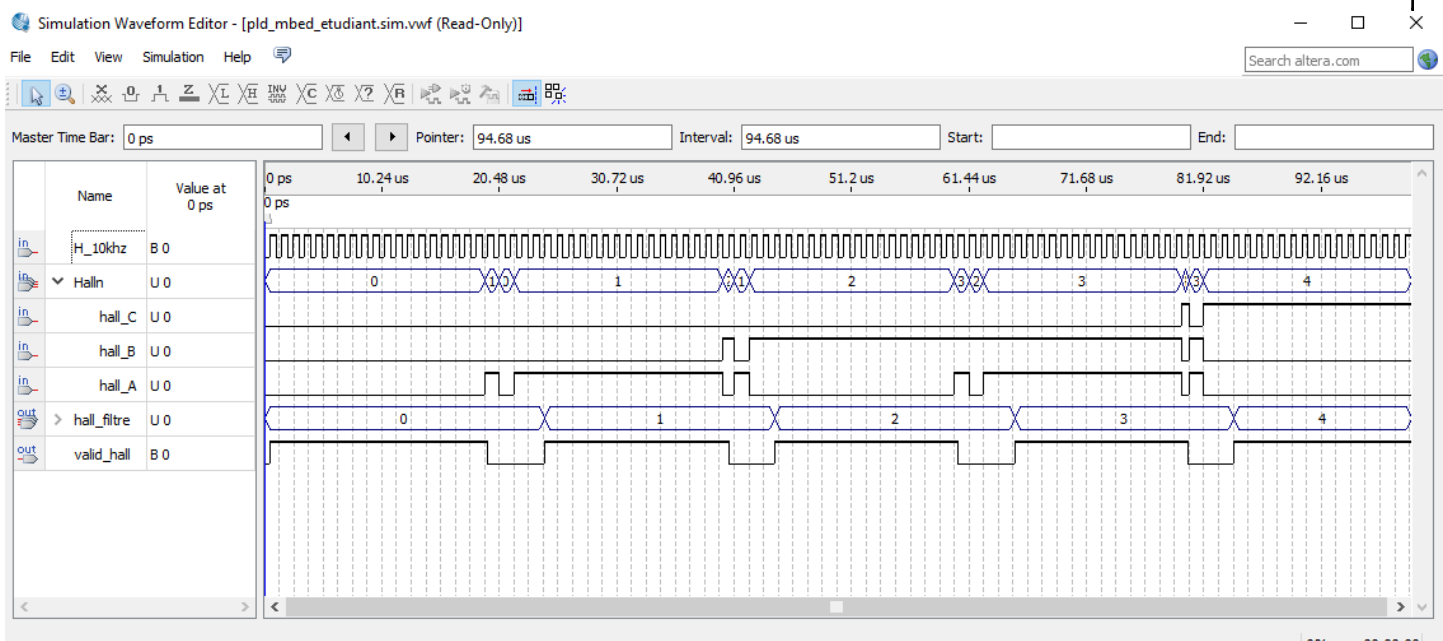
```
end if;
```

```
end if;
```

```
end process;
```

```
end archi;
```

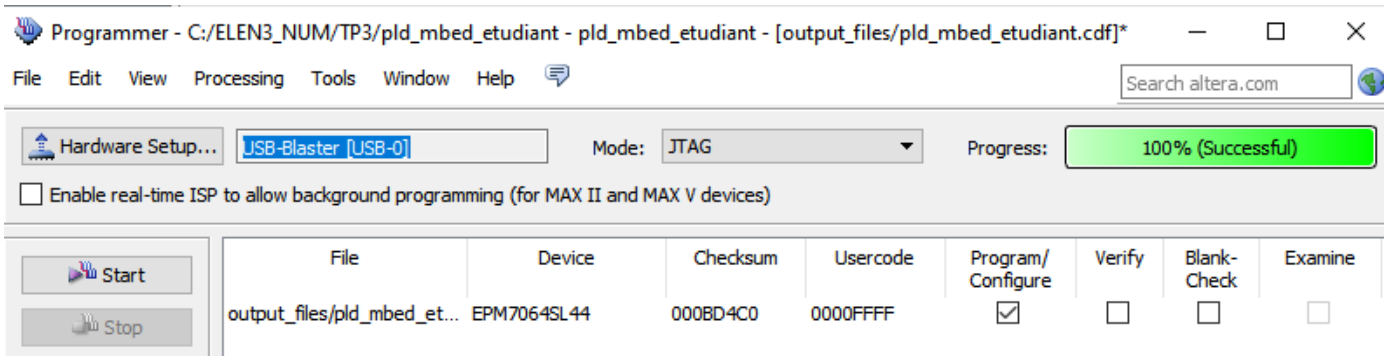
Collez ici votre simulation fonctionnelle FILTRE.vwf



- Pouvez-vous retrouver les effets de notre bloc de filtrage entre l'entrée brute HallIn[2..0] et la sortie filtrée Hall_filtre[2..0]

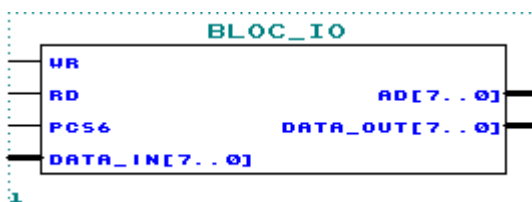
Etape de vérification n°1 avec une réf pwm constante:

- Placer une constante 8 bits = 127 sur l'entrée REF_PWM du bloc collecteur_num, et on validera avec une entrée constante à VCC en permanence l'entrée Valid_Pwm du bloc générateur PWM pour ce test.
- Terminer les interconnexions sur le fichier général d'interconnexions matrix_scooter_pwm_mbed_etudiant.bdf, le placer en top level entity et lancer une compilation générale
- Programmez le PLD Altera 7064SLC44-10 avec le fichier **pld_mbed_etudiant.pof** issue de la compilation générale.



- Câblez la maquette SAE roue sur le moteur brushless de la roue de vélo ou de scooter et tester le fonctionnement pratique de l'autopilotage . La roue doit se mettre à tourner.

3.4 Bloc de communication PLD/uC Mbed



Rôle : ce bloc (qui est entièrement fourni en code source VHDL) permet l'échange bidirectionnel de mots de 8 bits // entre le PLD Altera et le uC Beck.

En effet le PLD Altera est implanté sur le bus d'E/S // 8 bits du Mbed et donc accessible via la bibliothèque de fonction présente au projet port.c et port.h et la fonction PIO_Read()) le PLD lui retourne un vecteur de 8

bits DATA_IN[7..0].

De la même façon, si le uC tente une écriture dans cette plage d'adresse (PIO_Write()), la valeur écrite se retrouve disponible dans le PLD sur le vecteur DATA_OUT[7..0].

w

4 Etapes de travail à faire valider en séance

4.1 Autopilotage de la roue à partir du PLD Altera seul et d'une consigne de PWM fixée en dur à l'intérieur de ce dernier.

Il s'agit de désigner l'architecture interne du PLD pour que ce dernier puisse permettre de faire tourner la roue en marche avant ou en marche arrière (à partir de la lecture du bit de Direction) à vitesse constante (ref_pwm fixée en dur à 127 puis 255 à l'intérieur du PLD).

Test de validation : faire vérifier le fonctionnement dans les deux sens et mesurer la vitesse de la roue en fonction de la tension d'alimentation DC de l'onduleur variant entre 0Vet 30V pour les deux consignes de REF_PWM. Etablir une caractéristique Vitesse=F(Tension Onduleur) pour les deux référence REF_PWM possible. Comparer à la caractéristique Vitesse=F(Tension Hacheur) dans le cadre d'un moteur à courant continu équivalent.

4.2 Autopilotage de la roue à partir du PLD Altera et d'une consigne de PWM fournie par le uC MBED à partir d'une saisie console utilisateur.

Cette fois on reconfigure le PLD ALTERA pour qu'il reçoive sa consigne REF_PWM en provenance du uC MBED via la liaison de communication 8bits bidirectionnelle déjà en place.

Coté MBED, importer le squelette du programme du projet MBED depuis l'environnement cloud via le lien suivant:

https://os.mbed.com/teams/ERS3-2015-2016/code/scooter_mbed_etudiant/

Une fois importé en tant que programme faire l'analyse des différentes parties du code de ce squelette (pour cela utiliser la fiche ressource disponible sur LMS UNICE dans le cours « SAE ROUE » à savoir la fiche 2.

https://lms.univ-cotedazur.fr/2022/pluginfile.php/363357/mod_resource/content/1/fiche%20n2%20squelette%20de%20programme%20mbed.pdf

L'objectif du programme qui sera implanté dans le uC MBED est de permettre, via un petit menu console série (IHM atelier voir figure 5-1) en liaison RS232, la saisie ou la visualisation des paramètres pour notre contrôleur de VAE.

Dans cette étape c'est uniquement la consigne de PWM qui devra être saisie par l'utilisateur ce qui caractérisera le MODE MANUEL. C'est un mode uniquement accessible en atelier via cette liaison console.

4.2.1 Mise en place d'un menu console à choix multiples :

Voici à quoi peut ressembler ce menu console au fur et à mesure de l'avancée du projet :

```
entrez votre choix:
1 :lecture poignee gaz
2 :lecture courant
3 :lecture Vbat
4 : lecture temperature
5 : visu vitesse m/h
6 : lecture pe
7 : offset poignee min
8 : offset poignee max
9 : saisir rapport cyclique (entre 0 et 1000)
10: valeur des hall: 5
q : quitter
.
```

Figure 4-1 : menu console permettant la commande et la visualisation des paramètres du VAE

Ce menu console sera géré en tâche de fond dans le Main(). On pourra en sortir en tapant la lettre 'q' (quit) ce qui provoquera la mise à 0 de la REF_PWM et la dévalidation générale du générateur PWM implanté dans le PLD. Pour cela vous utiliser le bit de liaison MBED>PLD qui se nomme VALID_PWM (voir tableau d'affectation des signaux en figure 3).

L'envoi de cette référence REF_PWM depuis le uC MBED vers le PLD Altera se fera en utilisant une API déjà présente dans le squelette du projet et détaillée dans la fiche ressource FICHE 3.

- Dans cette étape on n'implantera au niveau du menu console que la possibilité de saisir manuellement un rapport cyclique compris entre 0 et 255.

Test de validation : relever la caractéristique Vitesse de rotation=F(Consigne PWM saisie). La vitesse sera mesurable au moyen du tachymètre optique mis à disposition.

Faire valider les étapes d'initialisation et de clôture du fonctionnement général (gestion du signal VALID_PWM).

Faire valider la commande de la vitesse de rotation de la roue à vitesse variable à l'aide de la saisie console REF_PWM

4.3 Mise en place d'une mesure de la vitesse.

Il s'agira cette fois d'exploiter les informations des capteurs de position du moteur brushless pour obtenir une mesure précise de la vitesse de rotation et la mettre à disposition de l'affichage dans la console.

Pour cela vous disposez d'un signal Valid_HALL (voir tableau d'affectation en figure 3) et présent sur le MBED sur P22. Ce signal présente un front montant à chaque changement de secteur d'alimentation du moteur brushless.

Visualiser ce signal Valid_Hall sur le uC MBED (mesure à effectuer à l'oscilloscope avec une sonde) et mesurer la relation Fréquence Valid_Hall=F(vitesse de la roue)..

Connaissant les paramètres de construction de la roue et, le nombre de secteur d'alimentation différents nécessaire pour faire un tour électrique de roue, son nombre de paires de pôles par phase, son diamètre en déduire la période de comptage de ces top de Valid_Hall pour obtenir au minimum une précision de mesure de vitesse de 1KM/H soit pour le moteur de vélo soit pour le moteur de scooter.

Etablir ensuite la relation $Vitesse = F(nb_top)$ en utilisant une période de comptage de ces tops de 100ms (période qui sera implantée dans le logiciel final). Les paramètres électrique (p=nombre paires de pôles/phase) géométrique (diamètre de la roue) seront inscrits dans un fichier de paramétrage config_grX.txt et seront relus à chaque démarrage de l'application.

Test de validation : rajouter une option « mesure de vitesse » au menu console qui permette d'afficher la vitesse de la roue en m/H et faire valider la lecture du fichier de paramétrage et l'affichage de la vitesse.

4.4 Réalisation d'un programme de test pour le réglage de la Ref_PWM depuis la poignée de Gaz

Le contrôleur de VAE est capable de lire une consigne analogique représentant la consigne de vitesse de la roue. Cette entrée analogique Vgaz est disponible sur la broche P17 du uC MBED (voir figure 3) .

Réfléchir à une architecture logicielle susceptible de procéder à la lecture de cette consigne analogique en dehors du menu déroulant (ce dernier étant bloquant en attente d'un choix utilisateur)

Implanter une nouvelle option au menu déroulant « Mode Auto/Manu » permettant à l'utilisateur de choisir d'où vient la consigne REF_PWM qu'il faudra envoyer au PLD.

- Mode Auto (mode par défaut au démarrage) la consigne Ref_PWM est issue de la poignée de Gaz.
- Mode Manu (mode utilisé en atelier) : La consigne est fixée par l'utilisateur à l'aide d'une saisie console.

Test de validation : relevez avec un oscilloscope la variation de la tension Vgaz en fonction des positions extrêmes de la poignées de Gaz. Déterminer les deux positions extrêmes de Vgaz à savoir Vgaz_Min (poignée au minimum) et Vgaz_Max (poignée au maximum). Comparez les paramètres à ceux mesurés par les autres binômes pour estimer la dispersion de ces caractéristiques **MAX = 2.92V MIN = 0.760V**

En déduite la relation en mode Auto : $Ref_PWM_AUTO = F(Vgaz, Vgaz_min, Vgaz_max)$ et la représenter graphiquement. Pensez à intégrer à votre logiciel une saturation de la valeur Ref_Pwm_AUTO afin de garantir que la consigne envoyée au PLD restera bien entre 0 et 255.

Pensez à gérer le capteur de frein pour couper instantanément la consigne d'assistance électrique en cas de freinage.

Faire valider le fonctionnement du mode auto et le basculement entre mode auto et mode manu sans gestion du frein

Faire valider le fonctionnement du mode auto avec la gestion du frein (la consigne REF_PWM doit tomber à 0 quelque soit la position de la poignée de Gaz si le frein est actif.

4.5 Réalisation d'un programme de calibration persistante de la poignée de Gaz

En complément de l'étape 4.5 on rajoutera une option au menu déroulant permettant de relever et archiver les paramètres de calibration de la pignée de Gaz à savoir Gaz_Min et Gaz_Max. ces paramètres seront acquis et affichés pendant la procédure de calibration réalisée par l'utilisateur grâce au menu console puis seront automatiquement archivés dans le fichier de config déjà mis en place à l'étape 4.3.

A chaque redémarrage de l'application ces paramètres seront restaurés dans l'application à partir du fichier de configuration et utilisés dans le code pour le conditionnement de la poignée de Gaz

Test de validation : faire valider la procédure de calibration de la poignée de gaz et la restauration des paramètres gaz_Min Gaz_Max à chaque démarrage de l'application.

4.6 Mise en place d'une commande progressive de la REF_PWM à partir de la poignée de Gaz(option)

On désire rendre plus progressif les variations de la consigne de Gaz . Pour cela on suppose que si la poignée passe de 0% à 100% la montée de la ref_pwm entre 0 et 255 devra se faire en 1,27s. ça permettra à l'utilisateur de retrouver des sensations qui connaît déjà sur un véhicule thermique ou l'inertie naturelle des pièces mécaniques en mouvement ralentis la dynamique de variation du couple moteur.

Si la consigne Ref_PWM_Brute diminue brusquement, la Ref_PWM_Finale doit la suivre instantanément. Si au contraire la consigne Ref_PWM_Brute augmente brusquement alors la REF_PWM_Finale doit remonter progressivement dans le temps. On désire toujours couper l'assistance électrique en cas de freinage, mais la remontée de la consigne Finale après un freinage doit être progressive.

Réfléchir et représenter le chronogramme de fonctionnement attendu de cette commande progressive ref_pwm_Final en fonction des variations instantanées de la Ref_PWM_Brute (variation instantanée de la consigne brute issue de la poignée) et d'un appui éventuel sur le levier de freinage.

4.7 Affichage du registre interne d'état présent dans le PLD Altera.

Le PLD altera contient un registre d'état internet sur 8 bits contenant des indicateurs représentant :

- Le numéro de secteur d'alimentation actuellement présente dans le moteur brushless (Hall[2..0])
- La direction choisie pour le sens de rotation de la roue (AV=1, AR=0)
- Un indicateur de défaillance FLTA
- Un indicateur d'appui sur le levier de frein Brake
- Un indicateur de surintensité Over_Current.

Voir ci-dessous la position de ces indicateurs dans le registre d'état.

DATA_IN0	--	HALLA
DATA_IN1	--	HALLB
DATA_IN2	--	HALLC
DATA_IN3	--	DIRECTION (1=AV 0=AR)
DATA_IN4	--	FLTA (actif=1)
DATA_IN5	--	BRAKE* (actif =0)
DATA_IN6	--	OVER_CURRENT* (actif=0)

Ce registre d'état peut être lu par le ucMbed au moyen d'une fonction de lecture présente dans l'API fournis et permettant la communication entre le uC et le PLD (plus de détail dans la fiche ressource FICHE 5).

Test de validation : rajouter une nouvelle option au menu console « affichage registre d'état » qui permette d'afficher en clair : Le numéro de secteur d'alimentation entre 0 et 7, la direction Avant/Arrière, l'activité de l'indicateur FLTA (ON/OFF) , l'activité du frein (ON/OFF), l'activité de l'indicateur de sur intensité (ON/OFF).

Faire valider cette affichage en fonction de la rotation de la roue à la main et l'action sur les indicateurs.

.....

4.8 Mise en place d'un bridage en vitesse :

Gere une limitation en vitesse (bride) réglable depuis la console série et sauvegardée de manière persistante dans le fichier de configuration générale du Mbed. Cette limitation en vitesse aura pour objectif de couper instantanément la consigne d'assistance électrique (agissant sur le générateur PWM présent dans le PLD) . Il s agit donc de comparer cycliquement (toutes les 100ms par exemple) la vitesse mesurée par le micro contrôleur à cette valeur de bride en vitesse afin de décider si la consigne d'assistance électrique liée à la poignée de gaz doit être maintenue ou pas.

4.9 Affichage texte des 4 valeurs analogiques disponibles sur le contrôleur:

Afficher les valeurs instantanées depuis la console série des valeurs de consigne de gaz (0 à 100%), tension batterie (0 à 60V), courant batterie (-5A à +5A), température du convertisseur (-25°C à +100°C).

4.10 Mise en place d'une page web dynamique de supervision.

Il s'agit cette fois d'utiliser les fonctionnalités de serveur http du uC MBED afin de collecter des informations temps réel sur le fonctionne de notre contrôleur (vitesse, niveau poignée de gaz, vbat, lbat, température...) et de l'afficher dans une page web lors de la connexion d'un navigateur client sur notre plateforme.

4.11 Mise en place d'une gestion de jauge d'énergie restante dans la batterie.

Le but cette fois est de fournir à l'utilisateur une jauge d'énergie basée sur la méthode de coulomb (comptabilisation des intensités entrantes et sortantes de la batterie) pour fourni une estimation du % d'énergie restante.

Cette méthode exploitera le capteur de courante batterie présente dans le contrôleur de puissance. La capacité initiale de batterie (Bat_Nominale) et la capacité restante de batterie (Bat_Restante) seront sauvegardés à la fin de l'application dans le fichier de sauvegarde persistante du Mbed afin de pouvoir être restaurés. Une option du menu déroulant doit permettre de pouvoir fixer la valeur nominale de la capacité batterie (en mA.s) et une option option doit permettre de pouvoir recharger virtuellement à 100% la capacité restante.

4.12 Mise en place d'un bus CAN pour déporter la commande de Gaz.

Proposer de déporter la gestion de la consigne d'assistance au pédalage et de freinage via un réseau CAN BUS. La poignée sera alors connectée depuis un 2eme microcontrôleur MBED (CAN emeteur) relié en BUS Can au contrôleur de puissance (CAN recepteur) afin de réaliser une poignée de gaz d'un communicante. La calibration et les données persistantes de calibration resteront eux localisés dans le Mbed controleur du VAE, seule l'information brute de consigne de gaz sera déportée via le bus CAN.

4.13 Développement d'une application Android pour l'affichage des paramètres système (projet tutoré).

Basé sur le développement précédent il s'agit cette fois de programmer une petite application tournant sous plateforme mobile Android pour afficher les paramètres systèmes. On pourra soit utiliser le serveur WEB déjà en place (dans ce cas l'application web s'apparente à un client http).

5 Evaluation du travail

La note de SAE ROUE sera basée sur 4 critères

- 1) **Notes de validation des étapes demandées en séances et listées ci-dessous**
- 2) **Note d'efficacité/comportement en séance permettant de différencier le travail des étudiants au sein d'un même binôme**
- 3) **Note de dossier remis à la fin de la SAE regroupant les livrables attendus (code C commenté, architecture du PLD, programmation Android, fiche de calibration du contrôleur VAE et notice d'utilisation utilisateur.**
- 4) **Note finale du DS de synthèse.**

Listes des fiches cuisines proposées : (liste évolutive)

- Fiche 0 : éléments constitutifs d'un véhicule à assistance électrique
- Fiche1 : principe de l'autopilotage par CPLD d'un brushless
- [Fiche2 : squelette de projet mbed pour le démarrage du projet](#)
- Fiche 3 : utilisation d'une classe d E/S permettant la communication entre uC MBED et CPLD
- Fiche 4 : principe de mesure de la vitesse du moteur par l'utilisation des capteurs de position du moteur
- Fiche 5 : principe de la sauvegarde et de la récupération de donnée persistante par l'utilisation d'un fichier texte
- Fiche5 : utilisation d'une bibliothèque permettant la génération d'un page web dynamique
- Fiche 6 : utilisation basique d app inventor pour la collecte et l'affichage de données contenues dans une page web
- Fiche 7 : principe de l'utilisation d'un bus CAN pour la communication de paramètres

6 La gestion de projet et l'évaluation

La SAE poursuit deux objectifs complémentaires :

- L'acquisition et la mise en œuvre de compétences qui doivent permettre la finalisation du projet. Mais il se peut que vous n'utilisiez pas l'ensemble ces connaissances pour finaliser le projet. Néanmoins on vous évaluera en DS sur la maîtrise de toutes ces connaissances
- La réalisation du projet. Il faut qu'un prototype fonctionne à l'issue de la SAE même s'il ne satisfait pas toutes les contraintes énoncées et à faire valider au fur et à mesure du développement.

6.1 Délivrables

- Un prototype fonctionnel avec les fiches de mesure et le programmes C commenté du Mbed ainsi que le détail et les simulation de la partie logique programmable .
- Rédaction en anglais/français d'une datasheet décrivant les principales caractéristiques matérielles du prototype et ses performances. Une notice d'utilisation du menu console atelier (calibration paramétrage) et un manuel d'utilisation grand public.